Agent Memory

DSC 204A: Scalable Data Systems

Haojian Jin

Where we are in the class?

Foundations of Data Systems

- Data representation & encoding
- Basics of Computer Organization
- Operating Systems
- Storage & Memory Hierarchy & Cache
- **Agent Memory**

Scaling & Distributed Systems

- Encoding & evolution
- Distributed Processing
- Programming model

Machine Learning Systems

- ML Systems
- Batch & Stream Processing

Spoiler Alert

1. What is memory in AI Agents?
2. What is prompt engineering?
3. What is context engineering?
4. How does OpenClaw work?
5. What are the differences between Claude Code and OpenClaw?
6. What are the differences between Claude Code and the Vanilla API?
7. Why is the memory not infinite?
8. How does Claude.md work? The memory hierarchy for agents.

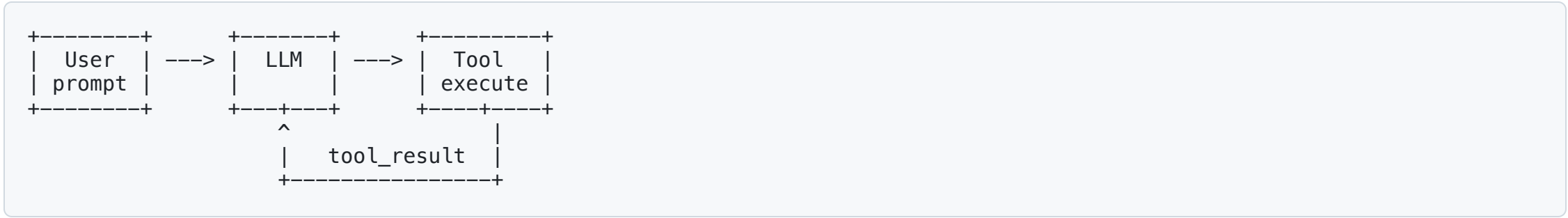
Part 0

What is an AI Agent?

What is an AI Agent?

An AI agent is a system that uses an LLM as its **reasoning engine** to decide what actions to take.

The core idea: **LLM + Tools + Loop**



> A chatbot generates text. An agent **takes actions** in the real world — reading files, running code, calling APIs, writing emails — and uses the results to decide what to do next.

Diagram adapted from learn.shareai.run, s01: The Agent Loop.

The Agent Loop: Send, Execute, Feed Back, Repeat

The entire architecture fits in ~30 lines of Python:

```python
def agent_loop(query, tools):
    messages = [{"role": "user", "content": query}]

    while True:
        response = llm.call(messages, tools)
        messages.append({"role": "assistant", "content": response})

        if response.stop_reason != "tool_use":
            return response.text  # Done — no more tools needed

        # Execute each tool the model requested
        results = []
        for tool_call in response.tool_calls:
            output = execute_tool(tool_call.name, tool_call.input)
            results.append({"tool_use_id": tool_call.id,
                            "content": output})

        messages.append({"role": "user", "content": results})
        # Loop continues — model sees results, decides next step
```

The "write-back" step is the key insight: tool results feed back into the conversation, enabling multi-step reasoning.

Adapted from learn.shareai.run, s01: The Agent Loop.

Tools Make the Agent

Without tools, an LLM is just a text generator. Tools give it **hands**.

Tool	What It Does	Why It Matters
bash	Execute shell commands	Run tests, install packages, git operations
read_file	Read file contents	Understand existing code
write_file	Create new files	Generate code, configs
edit_file	Modify existing files	Fix bugs, refactor
web_search	Search the internet	Find documentation, examples

Adding a new tool = one handler function + one schema entry. The loop doesn't change.

```
TOOL_HANDLERS = {
    "bash":       lambda **kw: run_bash(kw["command"]),
    "read_file":  lambda **kw: run_read(kw["path"]),
    "write_file": lambda **kw: run_write(kw["path"], kw["content"]),
    "edit_file":  lambda **kw: run_edit(kw["path"], kw["old"], kw["new"]),
}
```

Adapted from learn.shareai.run, s02: Tool Use.

Part 1

What is Memory in AI Agents?

AI Coding Agents: The State of the Art (2026)

Before we talk about memory, let's see where we are:

Metric	Number	Source
Developers using AI coding tools	**92%** (any workflow)	Developer Survey 2026
Engineers using AI tools **daily**	**73%** (up from 18% in 2024)	Developer Survey 2026
GitHub Copilot users	**20M** cumulative, 4.7M paid	GitHub, Jan 2026
Claude Code weekly active users	**1.6M**	Anthropic, 2026
OpenClaw GitHub stars	**356K** (most-starred project on GitHub)	GitHub, Mar 2026
% of code written by AI	**~50%** for Copilot users (61% for Java)	GitHub, 2025

The question isn't whether agents work — it's how they manage information to work well. That's today's topic.

The Core Problem: Agents Are Stateless

Every time you start a new chat with ChatGPT or Claude, the model has **zero memory** of your past conversations.

- No idea who you are
- No idea what you asked before
- No idea what it already told you

The model's **only knowledge** is:

1. Training data (frozen at cutoff date)
2. Whatever is in the **current context window**

```
Session 1:
User: "My name is Alice"
AI: "Hi Alice!"

Session 2:
User: "What's my name?"
AI: "I don't know your name."
```

Without memory, every session starts from scratch.

Human Memory vs. AI Agent Memory

Human Memory	What It Stores	AI Agent Equivalent	Implementation
Working memory	Active thoughts (~7 items)	**Context window**	Current prompt + messages
Episodic memory	Personal experiences	**Interaction logs**	Vector DB of past sessions
Semantic memory	Facts & knowledge	**Learned preferences**	Knowledge graphs, structured DB
Procedural memory	How to do things	**Tool-use patterns**	Model weights, prompt templates

> The key insight: **human working memory holds ~7 items**. An LLM context window holds up to **1M tokens**. But both are **finite** and **lossy**.

Baddeley, A. Working Memory. Science, 1992. | Miller, G. The Magical Number Seven. Psychological Review, 1956. | Sumers et al. Cognitive Architectures for Language Agents. 2023.

Case Study: Generative Agents (Stanford, 2023)

25 AI agents in a virtual town, each with:

1. **Memory Stream** — observations in natural language
2. **Retrieval** — scored by recency + importance + relevance
3. **Reflection** — summaries when importance threshold is hit

Result: One agent planned a Valentine's party. Invitations **spread organically**. Agents coordinated schedules — all from a **single seed instruction**.

```
Memory Stream:
[8am]  Klaus sees Maria at cafe
[8:05] Klaus tells Maria about party
[9am]  Maria tells John about party

Reflection (auto-generated):
"Klaus is socially active and enjoys
 organizing community events"
```

Park et al. Generative Agents: Interactive Simulacra of Human Behavior. UIST 2023.

MemGPT: The OS Analogy

Operating System

- **RAM** – fast, limited
- **Disk** – slow, large
- **Virtual memory** – OS pages data in/out of RAM transparently
- Programs see "infinite" memory

MemGPT Agent

- **Context window** – fast, limited
- **External storage** – slow, large
- **LLM uses function calls** to page data in/out of context
- Agent sees "unlimited" memory

> The insight: treat the LLM context like RAM, and use **self-directed paging** to create the illusion of unlimited memory.

Packer et al. MemGPT: Towards LLMs as Operating Systems. 2023. (Now the Letta framework)

Memory in Production Today

Feature	ChatGPT Memory	Claude Memory	Gemini Memory
Launched	Feb 2024	Sept 2025	Late 2024
How it works	Auto-extracts preferences & facts	Summarizes conversations	Auto-learns preferences
User control	View, edit, delete memories	View, toggle individual memories	Manage in settings
Persistence	Across all sessions	Across all sessions	Across sessions

The unsolved problem: When should episodic memory ("user corrected date format on Jan 5, 12, Feb 1") consolidate into semantic knowledge ("user prefers DD/MM/YYYY")?

Current systems use crude heuristics. This is an active research area.

How Claude Code Stores Memories

In practice, Claude Code uses **four memory categories** stored as individual markdown files:

Type	Purpose	Example
user	Stable preferences	"Prefers pnpm over npm", "Wants concise answers"
feedback	Enforced corrections	"Don't modify test snapshots", "Ask before codegen"
project	Non-obvious repo facts	"Legacy dir has deletion constraints", "Compliance requirements"
reference	External resource pointers	"Incident board at linear.app/INGEST", "Oncall dashboard URL"

```
# .memory/prefer_pnpm.md
---
name: prefer_pnpm
description: User prefers pnpm over npm
type: user
---
User explicitly asked to use pnpm for all package management.
```

> **"Memory gives direction; current observation gives truth."** — A memory that says "use React 18" might be stale. Always verify against the current codebase.

Adapted from learn.shareai.run, s09: Memory System.

Hands-on: Design an Agent's Memory

Exercise: You're building a **coding tutor agent** for DSC 204A. Design its memory architecture.

Memory Type	What to store?	Where?	Why?
Working	?	Context window	?
Episodic	?	?	?
Semantic	?	?	?
Procedural	?	?	?

Constraints: The context window is 200K tokens. The course has 300 students, each with different backgrounds. The agent must remember past mistakes it made for each student.

Discussion: Where does student-specific memory live? What happens when a student asks about a topic from 3 weeks ago?

> **Takeaway**: All agent memory flows through one bottleneck — the **context window**. Everything else manages what enters and exits this finite space.

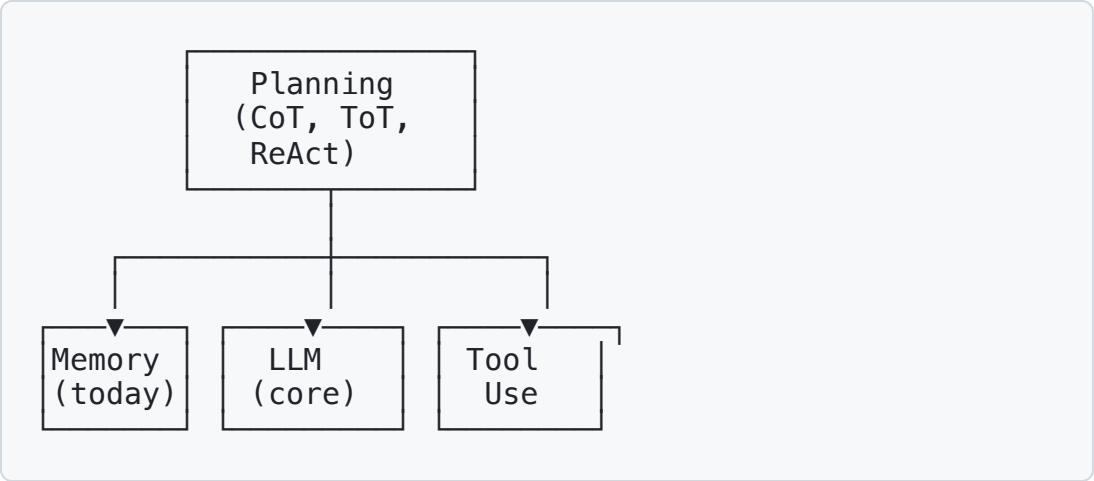
What Separates an Agent from a Chatbot?

Lilian Weng (OpenAI) identified three pillars of LLM-based agents:

1. **Planning** — break complex tasks into steps, reflect and refine
2. **Memory** — short-term (context) + long-term (retrieval)
3. **Tool Use** — interact with external world (APIs, code, search)

Andrew Ng added a fourth:

4. **Multi-agent collaboration** — specialized agents working together



Today we focus on **memory**. Planning and tool use are intertwined with how we manage context.

Do Agents Even Need Memory? Workflows vs. Agents

Workflow = deterministic orchestration

- You define the steps in code
- LLM handles each step, but flow is fixed
- More predictable, easier to debug

Agent = autonomous decision-making

- LLM decides what to do next
- Can take any number of steps
- More flexible, harder to control

Anthropic's advice: Start simple.

```
Level 0: Single LLM call
Level 1: Prompt chaining (A → B → C)
Level 2: Routing (classify → dispatch)
Level 3: Orchestrator-worker pattern
Level 4: Full autonomous agent
```

> "Most production use cases need **workflows**, not fully autonomous agents."

Why this matters for memory: Workflows need minimal memory (each step is independent). Agents need full context engineering (decisions depend on accumulated history).

Anthropic. Building Effective Agents. Dec 2024.

Part 2

What is Prompt Engineering?

Before Instruction Tuning: LLMs Were Autocomplete

Early LLMs (GPT-2, original GPT-3) were trained purely on **next-token prediction**:

> Given "The capital of France is", predict the next token → "Paris"

The problem: Ask "What is the capital of France?" and you might get:

```
Input:  "What is the capital of France?"
Output: "What is the capital of Germany?
         What is the capital of Italy?
         What is the capital of Spain?"
```

The model **continues the pattern** (a list of questions) instead of **answering**. It learned to predict text, not to follow instructions.

Instruction Tuning: Teaching LLMs to Follow Orders

Instruction tuning (2022) fine-tunes a base LLM on (instruction, response) pairs:

```
Instruction: "Summarize this article
              in 3 bullet points"
Response:    "• Point 1...
              • Point 2...
              • Point 3..."
```

+ RLHF (Reinforcement Learning from Human Feedback): humans rank outputs, model learns which responses humans prefer.

Before (base model):

- Completes patterns, not instructions
- Prompt engineering is the **only** way to steer it
- "Few-shot" = show examples so it mimics the pattern

After (instruction-tuned):

- Follows instructions naturally
- Much less prompt engineering needed
- But careful prompting **still helps significantly**

Ouyang et al. Training Language Models to Follow Instructions with Human Feedback (InstructGPT). NeurIPS 2022.

Why Prompt Engineering Still Matters

Even with instruction-tuned models, **how you ask** determines output quality:

Model	Simple Prompt	Engineered Prompt	Gap
GPT-4 on MMLU	~83%	~86%	+3%
Claude on SWE-bench	~33%	~49%	**+16%**
PaLM on GSM8K	17.7%	**78.7%** (CoT)	**+61%**

Instruction tuning made models understand "summarize this." **Prompt engineering** makes them do it **well** — with the right structure, depth, and format.

> The better the model, the less you need to fight it — but the more you can unlock from it.

What is Prompt Engineering?

The practice of designing natural language inputs to LLMs to get better outputs.

The term emerged ~2020-2021 as GPT-3 showed that **how you ask** matters as much as **what you ask**.

```
Bad prompt:
"Summarize this."

Better prompt:
"Summarize the following article in 3
bullet points, focusing on the key
technical contributions. Use simple
language suitable for a blog post."
```

The evolution:

1. **2020**: Zero-shot ("just ask")
2. **2021**: Few-shot ("show examples")
3. **2022**: Chain-of-thought ("think step by step")
4. **2023**: ReAct, Tree-of-Thought
5. **2024**: DSPy, meta-prompting
6. **2025**: Context engineering

Liu et al. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in NLP. 2023. | Brown et al. Language Models are Few-Shot Learners. NeurIPS 2020.

Zero-Shot vs. Few-Shot Prompting

Zero-shot — no examples provided

```
Classify the sentiment of this review:
"The battery lasts forever!"
→ Positive
```

Few-shot — provide examples first

```
"I love this!" → Positive
"Terrible quality." → Negative
"It's okay." → Neutral

"The battery lasts forever!" → ?
```

GPT-3 results (Brown et al., 2020):

Task	Zero-shot	Few-shot
SuperGLUE	55.4%	**71.8%**
LAMBADA	76.2%	**86.4%**
TriviaQA	64.3%	**71.2%**

Few-shot matched or exceeded **fine-tuned BERT** — without any gradient updates!

Brown et al. Language Models are Few-Shot Learners. NeurIPS 2020.

Chain-of-Thought: "Let's Think Step by Step"

Standard prompting:

```
Q: Roger has 5 tennis balls. He buys
2 more cans of 3. How many does he
have now?
A: 11   ✓ (sometimes)
```

Chain-of-thought prompting:

```
Q: Roger has 5 tennis balls. He buys
2 more cans of 3. How many does he
have now?
A: Roger started with 5 balls.
   2 cans × 3 balls = 6 balls.
   5 + 6 = 11.  ✓ (reliably)
```

The magic phrase: Adding "Let's think step by step" to prompts:

Benchmark	Standard	+ CoT
GSM8K (math)	17.7%	**78.7%**
MultiArith	17.7%	**78.7%**

That's a **4.4x improvement** from 6 words.

Why it works: Forces the model to use intermediate tokens as "working memory" for multi-step reasoning.

Wei et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022. | Kojima et al. Large Language Models are Zero-Shot Reasoners. NeurIPS 2022.

Tree-of-Thought: Exploring Multiple Paths

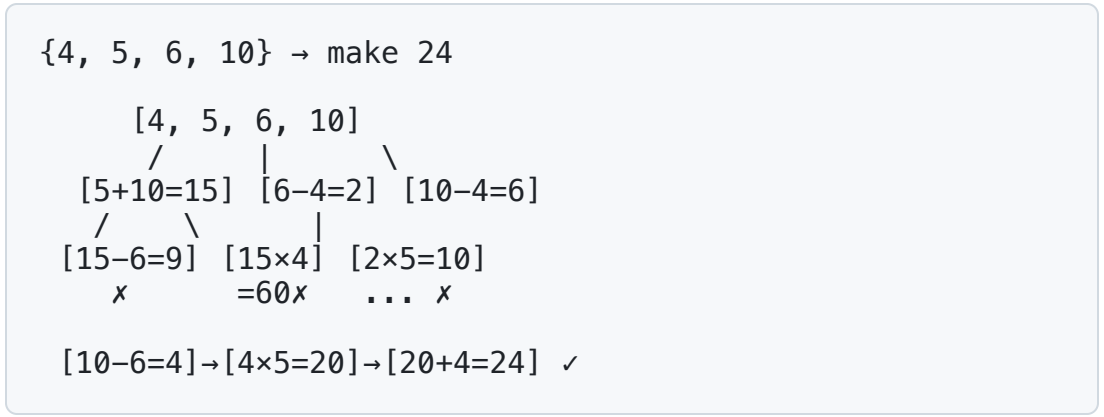
CoT = one path forward. **ToT** = explore multiple paths, evaluate, backtrack.

Like chess: consider moves, evaluate, prune.

Game of 24 (make 24 from 4 numbers):

Method	Success Rate
Standard	7.3%
Chain-of-thought	4.0%
Tree-of-thought	**74.0%**

CoT **fails** here because it can't backtrack.



Yao et al. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS 2023.

ReAct: Reasoning + Acting

The bridge between prompt engineering and agents. The model interleaves reasoning (thinking) with acting (using tools).

```
Q: "Elevation of the birthplace of the telephone inventor?"

Thought 1: I need to find who invented the telephone.
Action 1:  Search("inventor of the telephone")
Observe 1: Alexander Graham Bell.

Thought 2: Where was Bell born?
Action 2:  Search("Bell birthplace")
Observe 2: Edinburgh, Scotland.

Thought 3: What's Edinburgh's elevation?
Action 3:  Search("Edinburgh elevation")
Observe 3: ~47 meters.  →  Answer: 47 meters.
```

+6% on HotpotQA vs CoT, and critically reduced hallucination by grounding in retrieved facts.

Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.

The Brittleness Problem

Prompt engineering is **fragile**. Minor wording changes cause huge performance swings.

Sclar et al. (2024) tested semantically equivalent prompts on LLaMA:

> **Up to 76% accuracy variance** from changing prompt format alone — same meaning, different words.

Examples of brittleness:

```
Prompt A: "Is this positive or negative?"
→ 85% accuracy

Prompt B: "Classify sentiment as positive
           or negative."
→ 62% accuracy

Prompt C: "Sentiment: positive/negative?"
→ 78% accuracy
```

Same task. Same model. Same data.

Up to 23 percentage points difference.

> This is why prompt engineering alone isn't enough. We need something more **systematic**.

Sclar et al. Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design. ICLR 2024.

DSPy: Can We Automate Prompt Engineering?

DSPy (Stanford, 2024) compiles declarative LLM programs into optimized prompts automatically.

Instead of hand-crafting prompts:

```
# DSPy: Declare what you want
class QA(dspy.Module):
    def __init__(self):
        self.answer = dspy.ChainOfThought(
            "question -> answer"
        )
    def forward(self, question):
        return self.answer(question=question)

# DSPy optimizes the prompt for you
optimizer = dspy.BootstrapFewShot()
optimized_qa = optimizer.compile(QA(),
    trainset=examples)
```

The optimizer finds the best few-shot examples, instructions, and formats automatically.

Why this matters:

Prompt engineering is:

- **Brittle** – small changes break things
- **Model-specific** – GPT-4 prompts fail on Claude
- **Non-transferable** – can't share reliably

DSPy treats prompts as **learned parameters**, not hand-tuned strings.

> "The best prompt is the one a machine finds, not the one a human writes."

But – for agents, context engineering (not just prompts) is what matters.

Khattab et al. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. ICLR 2024.

Hands-on: Predict the Better Prompt

Exercise: For each task, two prompts are shown. **Predict which performs better** and **why**.

Task	Prompt A	Prompt B
Math (GSM8K)	"Solve: 23 × 17"	"Solve: 23 × 17. Show your work step by step."
Code bug	"Fix this code: `def f(x): return x/0`"	"You are a senior Python engineer. Review this code for bugs, explain the issue, then provide a corrected version: `def f(x): return x/0`"
Classification	"Is this email spam? [email text]"	"Classify: spam / not spam. Examples: 'Win $1M!' → spam. 'Meeting at 3pm' → not spam. Now classify: [email text]"

After predicting: Try both on Claude or ChatGPT. Were you right? Why?

Discussion: What technique does each Prompt B use? (Hint: each uses a different technique from the slides.) Is there a single "trick" that always works?

Part 3

What is Context Engineering?

The Evolution: Prompt → Context Engineering

> "+1 for 'context engineering' over 'prompt engineering'. People associate prompts with short task descriptions you'd give an LLM in your day-to-day use. In every industrial-strength LLM app, context engineering is **the delicate art and science of filling the context window with just the right information** for the next step."
>
> — **Andrej Karpathy**, June 2025

> "I really like the term 'context engineering' over prompt engineering. It describes the core skill better: **the art of providing all the context for the task to be plausibly solvable by the LLM.**"
>
> — **Tobi Lutke** (CEO, Shopify), June 2025

What is Context Engineering?

> **Context engineering** is the discipline of designing dynamic systems that provide the right information and tools, in the right format, at the right time, to give an LLM everything it needs to accomplish a task.

System Prompt	**Tool Definitions**	**Few-shot Examples**	**Message History**	**Retrieved Data**	**Agent Memory**
Role, rules, persona	Available actions	Desired behavior	Conversation state	RAG / search results	Persistent knowledge

The key shift: Context is not a static string — it's the **output of an engineered system** that runs before each LLM call.

Schmid, P. The New Skill in AI is Not Prompting — It's Context Engineering. 2025. | Anthropic. Effective Context Engineering for AI Agents. 2025.

Prompt Engineering vs. Context Engineering

Dimension	Prompt Engineering	Context Engineering
Focus	Craft the perfect instruction	Build the perfect **information environment**
Scope	One prompt string	Entire context window (system + tools + history + data)
Static vs. Dynamic	Usually static templates	Dynamic assembly at runtime
Who does it	The prompt author	An **automated system** (retrieval, memory, compaction)
Analogy	Writing a good email	Building a good **filing system**
Scale	Human-authored	Programmatic — can handle 200K+ tokens

> **Prompt engineering** is a **subset** of context engineering. It's one component — the instructions — in a much larger system.

The "Lost in the Middle" Problem

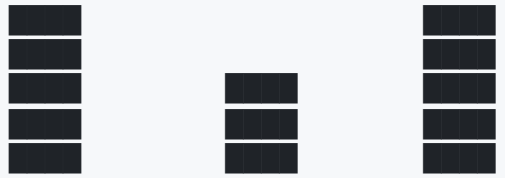
Liu et al. (2024): LLMs retrieve information placed at the **beginning** or **end** of the context reliably, but accuracy drops **10-20+ percentage points** for information in the middle.

This is a **U-shaped curve** — primacy and recency bias, just like human memory!

Implication for context engineering: You can't just "dump everything in." Where you place information matters as much as what information you include.

```
Accuracy by position in context:

Position:    Start  ...  Middle  ...  End
Accuracy:    85%         65%         82%
```



> Put critical instructions at the **top** and **bottom** of your context.

Liu et al. Lost in the Middle: How Language Models Use Long Contexts. TACL 2024.

Does Prompt Order Matter? A Pop Quiz

What's the difference between these two prompts?

(1) Instruction first, then content:

```
"Summarize the text."
+ [2000-word article]
```

The model reads "summarize" **first**, so it knows what to attend to as it processes the article. It has a **lens**.

(2) Content first, then instruction:

```
[2000-word article]
+ "Summarize the text."
```

The model processes 2000 words **without knowing why**. When it hits "summarize," it must rely on representations formed **without task intent**.

> Due to causal attention, when processing token N the model can only see tokens 1..N. In (1), "summarize" guides how article tokens are encoded. In (2), article tokens are encoded **generically**.

Why Instruction-First Wins (Usually)

The mechanism — causal attention:

- At token 500 in **(1)**: "summarize" is already in context — attention heads form **task-relevant representations**
- At token 500 in **(2)**: no task signal yet — representations are **generic**

Practical impact:

- Short content (~hundreds of tokens): difference is **minimal**
- Long content (~thousands of tokens): instruction-first is **measurably better**
- Smaller models: effect is **more pronounced**
- Large models (Claude, GPT-4): heavily trained on both orderings, gap has **narrowed**

The safe ordering for any prompt:

```
1. Instruction (what to do)
2. Context    (background info)
3. Input      (the actual data)
```

This connects directly to **primacy/recency bias**: models attend most to the **beginning** and **end** of context.

Best instruction placement:

- **Top** of prompt (primacy)
- **End** of prompt (recency)
- **Never** buried in the middle

Five Ways to Stretch a Finite Context Window

From Anthropic's engineering blog, five strategies for managing finite context:

1. Compaction — Summarize older conversation history when approaching limits

- Claude Code auto-compacts at ~98% capacity, reducing context by 60-80%

2. Structured note-taking — Agent writes persistent notes outside the window

- Retrieve only what's needed for the current step

3. Sub-agent architectures — Spawn focused agents with clean context windows

- Each sub-agent returns a condensed summary, not raw data

4. Token-efficient tool design — Tools return minimal but sufficient information

- Return 10 relevant lines, not the entire 5000-line file

5. Just-in-time retrieval — Load data dynamically via tools, not pre-stuffing

- Only retrieve what the current reasoning step actually needs

Anthropic. Effective Context Engineering for AI Agents. 2025.

How a System Prompt Is Assembled

The system prompt isn't a static string — it's **built by a pipeline** with 6 sequential stages:

```
class SystemPromptBuilder:
    def build(self):
        parts = []
        parts.append(self.core_identity())     # 1. Role, rules, safety
        parts.append(self.tool_catalog())       # 2. Available tools
        parts.append(self.skills())             # 3. Skill definitions
        parts.append(self.memory())             # 4. Recalled memories
        parts.append(self.claude_md())          # 5. CLAUDE.md chain
        parts.append(self.dynamic_context())    # 6. Date, cwd, mode
        return "\n\n".join(parts)
```

Stable content (stages 1-5) is cached across turns → prompt caching saves 90% cost.

Dynamic content (stage 6) changes per turn → always reprocessed.

> This is context engineering in action: a **program** decides what the model sees, not a human typing a prompt.

Adapted from learn.shareai.run, s10: System Prompt.

Subagents: Context as a Disposable Workspace

A subagent is a **context boundary**, not a process trick.

The problem: Exploring 5 files to answer "what testing framework?" dumps all file contents into the parent's context. Those artifacts stay forever.

The solution: Spawn a subagent with `messages=[]`. Only the **final summary** returns to the parent.

```
Parent context:
  [... existing work ...]
  → spawn subagent("find test framework")

Subagent context (fresh):
  [read file1] → [read file2] → [read file3]
  → "The project uses pytest"

Parent context (after):
  [... existing work ...]
  [tool_result: "The project uses pytest"]
```

5 file reads → 1 sentence. **Clean context.**

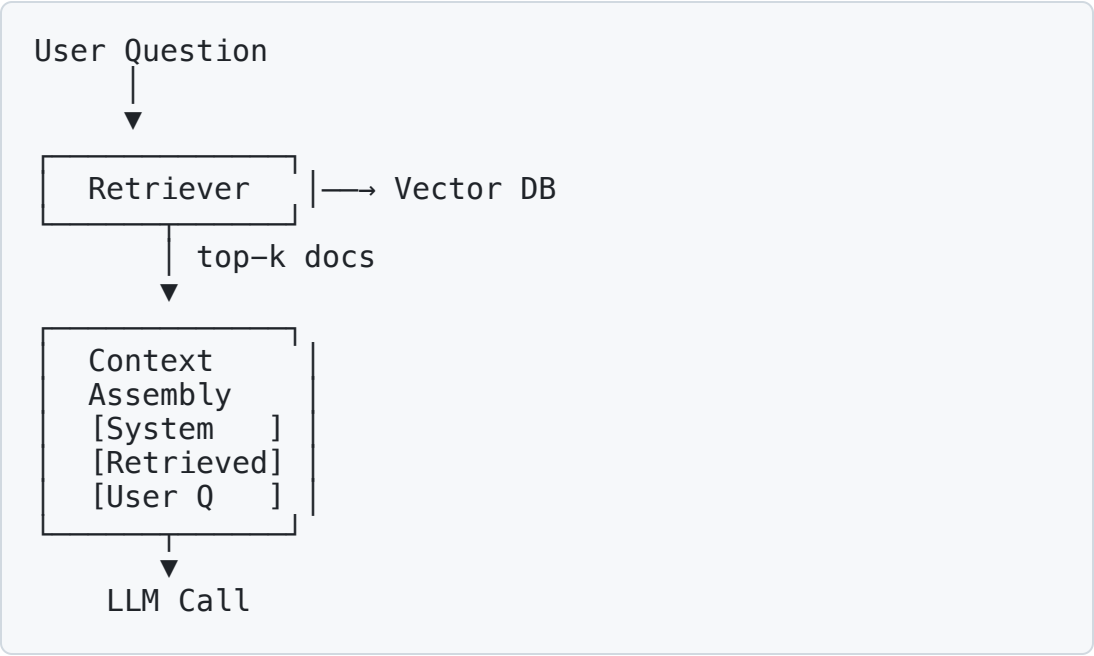
RAG Is Context Engineering

RAG (Retrieval-Augmented Generation):

1. User asks a question
2. System **retrieves** relevant docs from a knowledge base
3. Docs placed **in context** alongside the question
4. LLM generates answer **grounded in evidence**

RAG has evolved into a full **context engine**:

- Domain documents (traditional RAG)
- Conversation history (memory)
- Tool descriptions (capabilities)



A Real Agent in ~100 Lines: mini-SWE-agent

This agent scores **>74% on SWE-bench Verified** – better than most complex frameworks. The core is just context management:

```
class Agent:
  def __init__(self, model, system_prompt):
    self.history = [
      {"role": "system",
       "content": system_prompt}
    ]

  def step(self, observation):
    self.history.append(
      {"role": "user", "content": observation})
    response = model.chat(self.history)
    self.history.append(
      {"role": "assistant", "content": response})

    # Context engineering: trim if full
    if token_count(self.history) > MAX * 0.8:
      self.history = (
        [self.history[0]]     # keep system
        + self.history[-10:] # keep recent
      )

    return extract_bash_command(response)
```

What's happening here?

1. **System prompt** = persistent context (like CLAUDE.md)
2. **History** = working memory (grows each turn)
3. **Trimming** = compaction (evict old, keep recent)
4. **Observation** = tool results injected into context

No vector database. No fancy framework. Just **smart context management**.

The system prompt does the heavy lifting – it tells the agent how to explore codebases, write patches, and test fixes.

> The difference between a 30% and a 74% agent is **not** the model – it's how you **manage the context**.

SWE-agent/mini-swe-agent. GitHub, 2025.

Hands-on: Design a Context Budget

Exercise: You have **128K tokens** for a support agent. Allocate your budget.

Component	Tokens	%
System prompt	2,000	1.6%
Tool definitions (8 tools)	4,000	3.1%
Customer profile	1,000	0.8%
Recent tickets (last 3)	6,000	4.7%
Product catalog (RAG)	10,000	7.8%
Policy docs (RAG)	8,000	6.3%
Conversation history	50,000	39.1%
Output + buffer	**47,000**	**36.7%**

Discussion: What if a conversation runs long? What if the product isn't in top-5 RAG? How would you rebalance for a **coding agent** vs. a support agent?

Part 4

How Does OpenClaw Work?

What is OpenClaw?

An **open-source, general-purpose AI agent** that connects to messaging apps as a personal assistant.

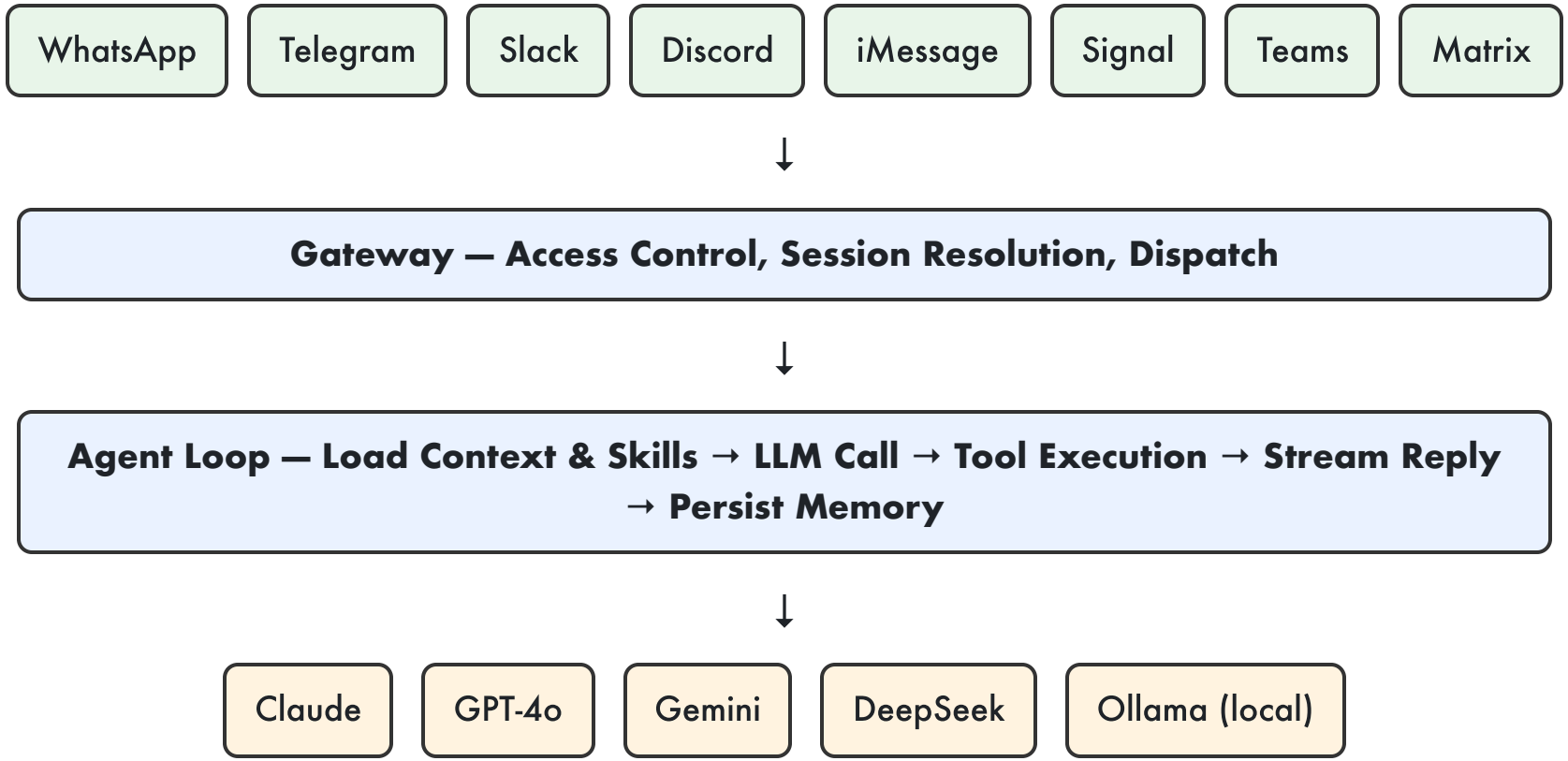
- Created by **Peter Steinberger** (PSPDFKit founder)
- Originally "Clawdbot" → "Moltbot" → **OpenClaw** (trademark issues)
- **356K+ GitHub stars** – most-starred project on GitHub
- Now maintained by a non-profit foundation

Why it went viral: Free, self-hosted, works with any messaging app you already use.

```
Nov 2025: "Clawdbot" launched
Jan 2026: Renamed "OpenClaw"
Feb 2026: 247K+ stars
          Steinberger → OpenAI
Mar 2026: 356K+ stars
          Non-profit foundation
```

OpenClaw GitHub: github.com/openclaw/openclaw | Wikipedia: OpenClaw

OpenClaw Architecture: The Gateway



Key design choice: The gateway pattern decouples messaging interface from model – you can swap LLMs without changing any skill or channel integration.

OpenClaw's Skills System

Skills are **markdown files** with YAML frontmatter.

On startup, OpenClaw scans skills, injects a compact list into the prompt, and loads specific skills **on demand**.

This is **lazy loading for AI context** – only consume tokens for skills you actually need.

```
# skills/email-assistant/SKILL.md
---
name: email-assistant
description: Draft and send emails
trigger: when user asks about email
---
## Instructions
1. Fetch context with Gmail tool
2. Draft response matching user tone
3. Stage for approval before sending

## Tools
- gmail_read(thread_id)
- gmail_draft(to, subject, body)
```

OpenClaw: What Can It Do?

Capabilities:

- Read/write files, run shell commands
- Browse the web, send emails
- Control APIs and external services
- 24/7 persistent agent runtime
- Subscribe to webhooks
- Respond to calendar events
- **20+ messaging platforms** as interface

Use cases:

- "Summarize my unread emails"
- "Schedule a meeting with the team"
- "Monitor this GitHub repo for new issues"
- "Remind me to buy groceries at 5pm"
- "Write and deploy a simple web scraper"
- "Track flight prices and alert me"

Key difference from voice assistants: open-source, self-hosted, and extensible via code — not a walled garden.

OpenClaw: Security Considerations

The same broad permissions that make OpenClaw useful make it an attack surface:

- **138+ CVEs tracked** — broad permissions draw security scrutiny
- Runs on your machine with access to your files, emails, APIs
- Security model relies on **access control at the gateway level**
- Community-contributed skills may contain vulnerabilities

The tradeoff:

More Capable	More Risky
Can control any API	Prompt injection could trigger unintended API calls
Reads your email	A compromised skill could exfiltrate data
Runs shell commands	Arbitrary code execution surface
24/7 operation	Persistent attack surface

> This is the **capability vs. safety tension** at the heart of all agent design.

CrowdStrike. What Security Teams Need to Know About OpenClaw AI Super Agent. 2026.

Part 5

Claude Code vs. OpenClaw

Two Philosophies of AI Agents

Claude Code — The Specialist

"Do one thing and do it well"

- **Purpose-built** for software engineering
- Terminal / IDE integration
- Deep codebase understanding
- Reads files, runs tests, commits code
- **Anthropic models only** (single-vendor lock-in)
- Simple install: `npm install -g @anthropic-ai/claude-code`

OpenClaw — The Generalist

"Do everything, everywhere"

- **General-purpose** life/work assistant
- 20+ messaging platforms
- Email, calendar, web, shell, APIs
- Connects to any service
- **Any LLM** (Claude, GPT, Gemini, local)
- Complex setup: Docker, Python, sandboxing

Feature Comparison

Feature	Claude Code	OpenClaw
Primary use	Software engineering	General-purpose assistant
Interface	Terminal, VS Code, JetBrains	WhatsApp, Slack, Telegram, etc.
Code editing	**Excellent** — diff views, multi-file	Has coding skills, not specialized
Context window	1M tokens (Opus 4.6)	Depends on model chosen
Memory system	CLAUDE.md files + auto-memory	Skills + persistent session storage
Model support	Anthropic only	Any provider
Setup	One command	Docker + configuration
Security	Permission-gated tools	Gateway access control
24/7 operation	No (session-based)	**Yes** — persistent daemon
Cost	API tokens (~$13/dev/day)	Free tool + API tokens
Open source	Partially (SDK open)	Fully open source

When to Use Which?

Use Claude Code when:

- Building or debugging software
- Refactoring a large codebase
- Writing tests and documentation
- Git workflows (commit, PR, review)
- You want deep code understanding
- You need reliable, safe file editing

Use OpenClaw when:

- Automating daily tasks across apps
- Managing email, calendar, messages
- Monitoring services and alerting
- Building personal workflows
- You want a 24/7 always-on assistant
- You want model flexibility

> **They're not competitors** — they solve different problems. You might use Claude Code for work and OpenClaw for personal automation.

Part 6

Claude Code vs. the Vanilla API

What is the "Vanilla" Claude API?

The Claude API is a **text-in, text-out interface**:

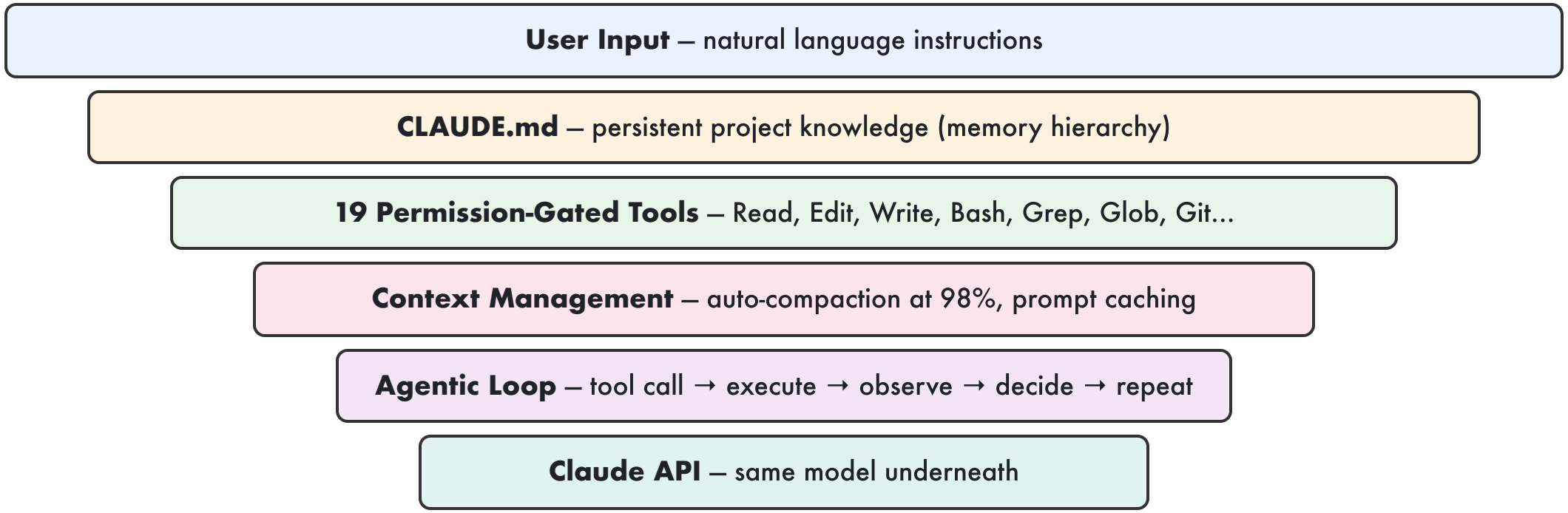
```
import anthropic
client = anthropic.Anthropic()
response = client.messages.create(
    model="claude-opus-4-6",
    max_tokens=1024,
    messages=[{"role": "user",
               "content": "Explain recursion"}]
)
print(response.content[0].text)
```

You send messages, you get text back. **No file access, no shell, no memory** unless you build it yourself.

You get: text generation, multi-turn chat, tool use (you define + execute), vision, prompt caching

You don't get: file system access, shell execution, git integration, permission management, auto-compaction, persistent memory, session management

What Claude Code Adds: The Agentic Harness



> The model is identical in both cases. The performance difference comes entirely from the **harness**: tools, memory, and the autonomous loop.

The Agentic Loop

Claude Code's core is a **single-threaded while-loop**:

```
while true:
    1. Send conversation history + tool definitions to Claude API
    2. Claude responds with text AND/OR tool calls

    if response has tool calls:
        3. Check permissions (auto-allow? ask user?)
        4. Execute each tool (read file, run bash, edit code...)
        5. Append tool results to conversation history
        6. Go to step 1 (let Claude see results and decide next step)

    if response is text only:
        7. Display to user
        8. Wait for next user input

    if context > 98% full:
        9. Auto-compact: summarize older messages, continue
```

Three phases per task: (1) Gather context, (2) Take action, (3) Verify results.

The Permission System

Claude Code gates every tool through a **cascading permission model**:

Level	Example	Default
Auto-allow (safe)	Read files, Grep, Glob	Always permitted
Session-allow	Edit files, Write files	Ask once, allow for session
Always-ask (dangerous)	Bash commands, Git push	Ask every time
Blocked	`rm -rf /`, force push to main	Denied with warning

```
Claude wants to run: git push origin feature-branch

Allow this action?
[y] Yes  [n] No  [a] Always allow git push  [d] Deny always
```

The principle: Safe operations auto-approve, dangerous ones require human consent. The human stays in the loop for consequential actions.

Hands-on: API Call vs. Claude Code

Task: "Fix the bug in auth.py where login fails for emails with '+' characters"

Vanilla API (you do the work):

1. You read auth.py manually
2. You paste code into the prompt
3. Claude suggests a fix as text
4. You manually apply the fix
5. You run tests manually
6. If tests fail, you paste errors back
7. Repeat until fixed

You manage: file I/O, context assembly, iteration

Claude Code (agent does the work):

1. You say: "Fix the login bug for + emails"
2. Claude Code reads auth.py (tool: Read)
3. Finds the bug (tool: Grep)
4. Applies the fix (tool: Edit)
5. Runs tests (tool: Bash)
6. Sees failure, reads error, fixes again
7. Tests pass → reports done

You manage: just the initial instruction

> The **same model** powers both. The difference is the **agentic harness** — tools, permissions, and the autonomous loop.

Part 7

Why Is the Memory Not Infinite?

The Dream: Infinite Context

Wouldn't it be nice if we could just dump **everything** into the context window?

- Entire codebase? Just paste it in.
- All past conversations? Keep them all.
- Every document in the company? Include them.

Why can't we?

Four fundamental constraints:

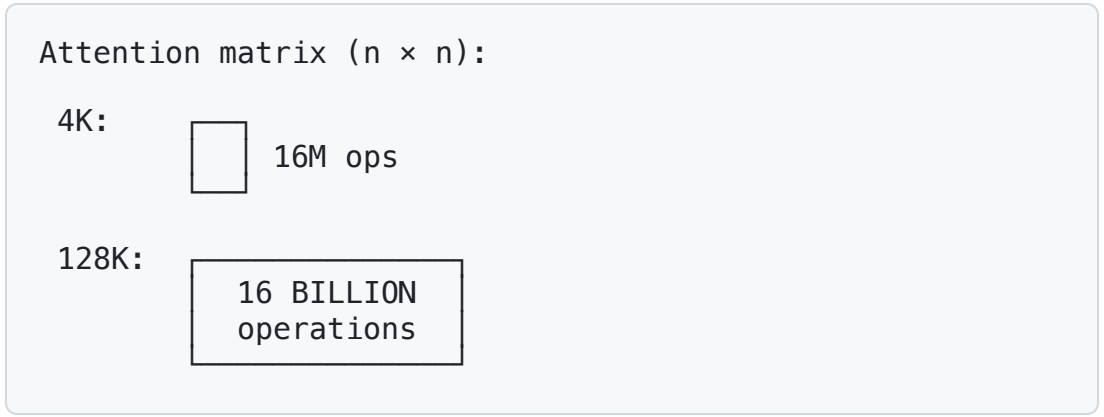
1. **Compute** — attention is $O(n^2)$
2. **Memory** — KV cache grows linearly per token
3. **Cost** — more tokens = more money
4. **Quality** — models get worse with more irrelevant context

Constraint 1: Quadratic Attention

Self-attention is **$O(n^2)$** in sequence length — every token attends to every other token.

Doubling context **quadruples** compute. (FlashAttention speeds I/O, but $O(n^2)$ FLOPs remain.)

Context Length	Relative Attention Cost
4K	1x
32K	64x
128K	1,024x
1M	62,500x



Vaswani et al. Attention Is All You Need. NeurIPS 2017.

Constraint 2: KV Cache Memory

During inference, every token's **Key** and **Value** vectors must be stored for every layer and head.

For a 70B-parameter model (e.g., Llama 3.1 70B):

- 80 layers × 64 heads × 128 dimensions × 2 (K+V) × 2 bytes (FP16)
- = **~2.6 MB per token**

Context Length	KV Cache Size	Hardware Needed
4K tokens	~10 GB	1 GPU
128K tokens	**~330 GB**	4-5 H100s (80 GB each)
1M tokens	**~2.6 TB**	33+ H100s

A single H100 GPU has **80 GB** of memory. You can't even fit the KV cache for 128K tokens on one GPU, let alone the model weights.

Pope et al. Efficiently Scaling Transformer Inference. MLSys 2023.

Constraint 3: The Cost of Context

More context = more money. Every input token costs real dollars.

Model	Price per 1M Input Tokens	Cost for Full Context
GPT-4o (OpenAI)	$2.50	$0.32 (128K)
Claude Opus 4.6	$5.00	$5.00 (1M)
Claude Sonnet 4.6	$3.00	$3.00 (1M)
Gemini 3 Flash (≤128K)	$2.00	$0.26 (128K)
Gemini 3 Pro (>128K)	$5.00	**$50.00** (10M) — long context premium

Latency also scales: A 128K-token prompt can take **30-60 seconds** just for prefill (before any output).

Throughput drops: Long-context requests monopolize GPU memory, reducing how many users the server can handle concurrently.

Constraint 4: Lost in the Middle

Even when you can **fit** the tokens, models struggle to **use** them.

Liu et al. (2024) showed:

- Information at the **beginning**: ~85% accuracy
- Information in the **middle**: ~65% accuracy
- Information at the **end**: ~82% accuracy

A **20-percentage-point drop** for mid-context information!

This is true even for models specifically designed for long contexts.

The Needle-in-a-Haystack test:
Place one key fact at varying depths in a long document. Can the model find it?

- **Gemini 1.5 Pro** (2024): Near-perfect on single needle
- **Claude 3** (2024): Near-perfect on single needle
- **Multi-needle tasks**: Performance degrades significantly

Simple retrieval works. **Reasoning over scattered context** does not.

Liu et al. Lost in the Middle. TACL 2024. | Kamradt, G. Needle in a Haystack Benchmark. 2023.

Current Context Windows (Spring 2026)

Model	Advertised	Effective*	Pages
GPT-4o (OpenAI)	128K	~70K	~175
Claude Opus 4.6 (Anthropic)	1M	~600K	~1,500
Gemini 3 Pro (Google)	10M	~5M	~12,500
Gemini 3 Flash (Google)	1M	~600K	~1,500

NVIDIA RULER benchmark: effective context is **50-65% of advertised.*

Sounds like a lot? The Linux kernel is ~28M tokens. A large codebase: 100M+. An agent's observation history grows **unboundedly**.

Research Frontiers: Extending Context

Approach	How It Works	Status
RoPE Scaling / YaRN	Extrapolate rotary position embeddings beyond training length	Extended LLaMA 2 from 4K → 128K
ALiBi	Linear bias on attention scores, no positional embeddings	Better length generalization
Ring Attention	Distribute sequence across GPUs in a ring topology	Near-linear memory scaling
FlashAttention	Fuse attention computation to avoid materializing full n×n matrix	2-4x speedup, same result
Mamba / SSMs	Replace attention with O(n) state-space models	No quadratic bottleneck
Mixture of Experts	Only activate subset of parameters per token	More efficient at scale

> These approaches reduce the **constant factors** or change the **complexity class**, but none eliminate the fundamental tradeoff: **more context = more resources**.

Peng et al. YaRN: Efficient Context Window Extension. 2023. | Dao et al. FlashAttention. NeurIPS 2022. | Gu & Dao. Mamba: Linear-Time Sequence Modeling. 2023.

The Analogy: Human Memory Is Finite Too

Human working memory:

- ~4-7 "chunks" (Miller, 1956)
- Decays in seconds without rehearsal

Compensation: long-term memory (lossy), external tools (notes, books), selective attention

LLM "working memory":

- Context window (up to 1M tokens)
- Disappears after the session

Compensation: RAG (external retrieval), memory files (persistent notes), context engineering (selective inclusion)

> Both humans and agents solve the same problem: **finite working memory + infinite world → selective retrieval**.

The Fundamental Tradeoff

More Context

+ More information available
+ Better continuity
+ Fewer retrieval errors

- More expensive
- Slower
- Lost in the middle
- More irrelevant noise

Less Context

+ Cheaper and faster
+ Focused on relevant info
+ Better signal-to-noise ratio

- May miss critical information
- Retrieval can fail
- Loss of continuity

> The optimal strategy is not "use all the context" or "use minimal context" — it's **use the right context**. That's context engineering.

Hands-on: Calculate the Cost

Exercise: Your agent makes 100 API calls per task, each using 50K input tokens with Claude Sonnet 4.6.

```
Cost per call:     50,000 tokens × $3.00 / 1,000,000 = $0.15
Cost per task:     100 calls × $0.15 = $15.00
Cost per day:      10 tasks × $15.00 = $150.00
Cost per month:    $150 × 22 work days = $3,300.00
```

Now with prompt caching (90% cache hit → 90% discount on cached tokens):

```
Cached tokens:     45,000 × $0.30/1M = $0.014
Uncached tokens:    5,000 × $3.00/1M = $0.015
Cost per call:     $0.029 (80% cheaper!)
Cost per month:    $638 (saved $2,662)
```

> **Prompt caching** is one of the most impactful context engineering techniques. Reuse the same system prompt, CLAUDE.md, and tool definitions across calls.

Part 8

How Does CLAUDE.md Work?

The Memory Hierarchy for Agents

Recall: The Computer Memory Hierarchy

From our previous lecture:

Level	Speed	Capacity	Cost	Persistence
Registers	~1 ns	~KBs	$$$	Volatile
L1/L2 Cache	~1-10 ns	~MBs	$$	Volatile
RAM (DRAM)	~100 ns	~GBs	$	Volatile
SSD	~100 µs	~TBs	¢	Persistent
HDD / Cloud	~10 ms	~PBs	¢¢	Persistent

Key principle: Faster memory is smaller and more expensive. Slower memory is larger and cheaper. We use **caching** to keep frequently-accessed data in fast memory.

Can we apply the same principle to AI agents?

The Agent Memory Hierarchy

Level	Agent Equivalent	Speed	Capacity	Persistence
Registers	Model weights (knowledge baked in)	Instant	Fixed at training	Permanent
L1 Cache	System prompt + CLAUDE.md	Loaded once	~10K tokens	Per-session
L2 Cache	Conversation history	Each turn	~200K-1M tokens	Per-session
RAM	Tool results (file reads, search)	On-demand	Varies	Per-call
SSD	Memory files, vector DB	Retrieval needed	Unlimited	Persistent
Cloud	Web search, external APIs	Network latency	The internet	Persistent

The same principle applies: Frequently-needed information (project rules, coding style) should live in "cache" (CLAUDE.md). Rarely-needed information (specific file contents) should be fetched on demand.

How CLAUDE.md Works

CLAUDE.md = **plain markdown** providing persistent instructions. Loaded automatically at session start — a cache that's always warm.

Four tiers that **accumulate** (not override):

1. **Managed policy** — org-wide, cannot be excluded
2. **Global** — `~/.claude/CLAUDE.md` (your prefs)
3. **Project** — `./CLAUDE.md` (team rules, in git)
4. **Local** — `./CLAUDE.local.md` (gitignored)

Plus: `.claude/rules/*.md` for **path-scoped rules**.

Loading order (first → last):

System Instructions	← Anthropic
Managed Policy	← Org admin
Global CLAUDE.md	← User prefs
Project CLAUDE.md	← Team rules
CLAUDE.local.md	← Personal
.claude/rules/	← Path-scoped
Auto-memory + History	← Dynamic

What Goes in a CLAUDE.md?

```
# My Project CLAUDE.md

## Build & Test
- Run tests: `npm test`
- Build: `npm run build`
- Lint before committing: `npm run lint`

## Code Style
- Use TypeScript strict mode
- Prefer functional components with hooks
- No classes unless interfacing with legacy code

## Architecture
- API routes in src/api/
- Components in src/components/
- State management: Zustand (not Redux)

## Important Context
- We deploy to AWS Lambda — keep bundle size small
- The auth system uses JWT with refresh tokens
- Never modify the migration files directly
```

This is loaded into every Claude Code session automatically. The agent **always** knows your project rules.

Similar Systems in Other Tools

The "persistent instructions" pattern is now universal:

Tool	Instruction File	Scope
Claude Code	`CLAUDE.md`, `.claude/rules/`	Global + Project + Local
GitHub Copilot	`AGENTS.md`, `.github/copilot-instructions.md`	Repository
Cursor	`.cursor/rules/*.mdc`	Project
Aider	`.aider.conf.yml` → `CONVENTIONS.md`	Project
ChatGPT	Custom Instructions (UI)	User-level
OpenClaw	`skills/*.md`	Skill-level

GitHub's analysis of **2,500+ repos** found 6 essential domains: **commands, testing, structure, code style, git workflow, boundaries**.

GitHub Blog. How to Write a Great agents.md – Lessons from Over 2,500 Repositories. 2025.

The Cache Analogy Goes Deep

Remember cache concepts from our hardware lecture?

Cache Concept	Agent Memory Equivalent
Cache hit	Info already in context → instant use
Cache miss	Info not in context → must retrieve (tool call, RAG)
Cold miss	First time accessing a file → must read it
Capacity miss	Context window full → must evict (compaction)
Write-back	Agent saves notes to memory files for later retrieval
Prefetching	CLAUDE.md pre-loaded at session start
Eviction policy	Compaction keeps system prompt + recent messages; summarizes the rest

> Agent memory management is **literally a caching problem**. The same principles from hardware memory hierarchy apply to LLM context management.

Four Levers of Context Compression

Claude Code uses a **4-lever strategy** — each lever triggers at different times:

Lever	When	What Happens
0: Persist to disk	Tool execution	Large outputs (>50KB) saved to disk, replaced with 2KB preview marker
1: Micro-compact	Every turn (silent)	Old tool results become `[Previous: used read_file]` placeholders
2: Auto-compact	Token threshold (~50K)	Full transcript saved to JSONL, LLM summarizes conversation, continues from summary
3: Manual compact	Agent decides	Agent triggers summarization at the optimal moment

```
Each API call:
  micro_compact() → check tokens → auto_compact if needed
  → API call → persist_output() → manual_compact if triggered
```

> **"Compaction isn't deleting history — it's relocating detail."** Full transcripts and outputs are preserved on disk. Only the active context is compressed.

Adapted from learn.shareai.run, s06: Context Compact.

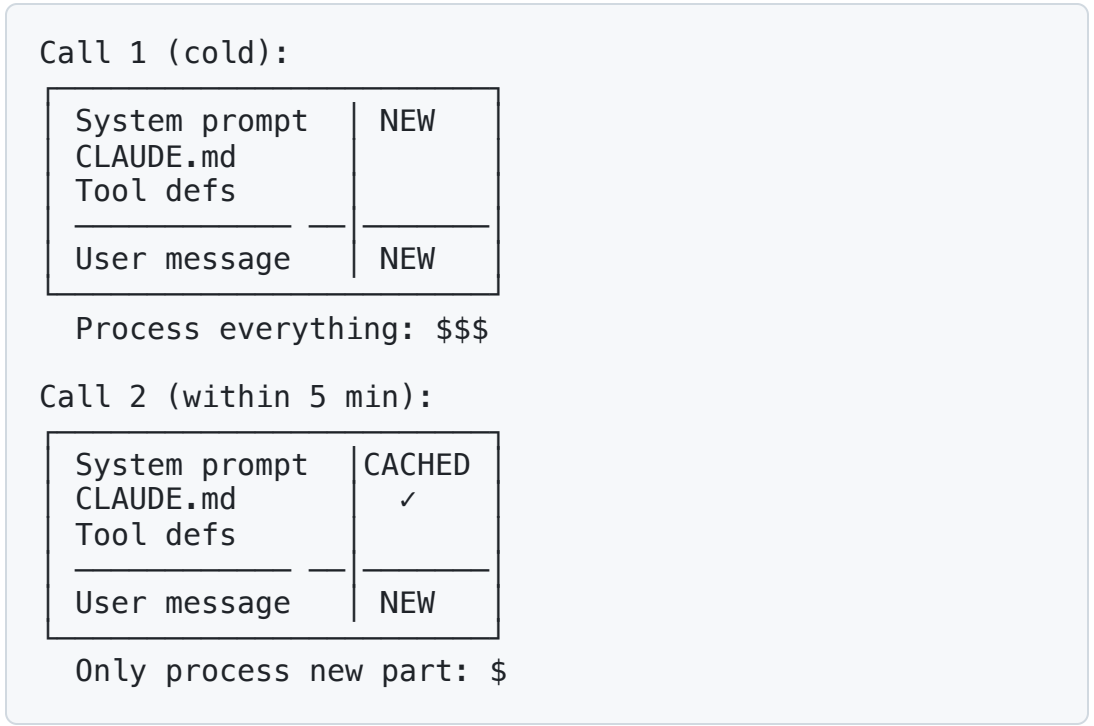
Prompt Caching: Speed and Cost

Problem: Every API call re-processes the full system prompt, CLAUDE.md, and tool definitions. That's ~10-20K tokens repeated identically.

Solution: Anthropic's **prompt caching** stores processed prefixes. Subsequent calls skip re-processing cached content.

Impact:

- **90% cost reduction** on cached tokens
- **85% latency reduction** on time-to-first-token
- Cache TTL: **5 minutes**



Hands-on: Writing a CLAUDE.md

Exercise: Write a CLAUDE.md for a data science project.

```
# DSC 204A Project — CLAUDE.md

## Environment
- Python 3.11, use pip (not conda)
- Run tests: `pytest tests/ -v`
- Data lives in `data/` — never commit data files

## Code Style
- Type hints on all functions
- Docstrings in Google format
- Use pathlib, not os.path
- Pandas for data manipulation, Polars for large datasets

## Architecture
- ETL pipeline: `src/etl/`
- Models: `src/models/`
- Notebooks are for exploration only — production code goes in `src/`

## Important Rules
- Never hardcode file paths — use config.yaml
- All experiments must be logged to MLflow
- GPU code must work on both CUDA and MPS (Mac)
```

Try it: Create this file in your project root. Claude Code will automatically pick it up.

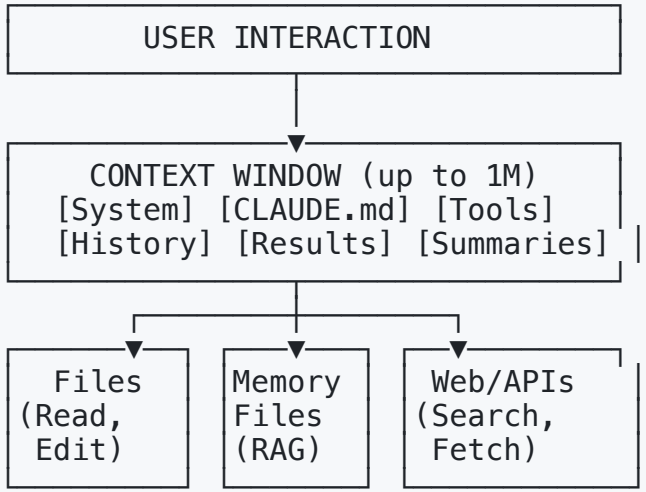
When the Memory Hierarchy Breaks

Just like hardware caches can fail, agent memory can go wrong:

Failure	Cause	Symptom	Fix
CLAUDE.md too large	>200 lines, kitchen-sink instructions	Model ignores late instructions (lost in the middle!)	Split into `.claude/rules/` with path scoping
Contradictory instructions	Global CLAUDE.md says "use tabs", project says "use spaces"	Inconsistent behavior	Audit across tiers, remove conflicts
Stale memory	Auto-memory remembers deleted patterns	Agent uses outdated approaches	Periodically clean memory files
Context thrashing	Agent reads many large files, compacts, re-reads	Slow, expensive, forgets earlier context	Use targeted reads (line ranges, grep first)
Cold start	No CLAUDE.md, no memory, no context	Agent asks basic questions every session	Write a good CLAUDE.md on day one

> Sound familiar? These are the **same failure modes** as hardware caches — capacity misses, coherence problems, cold starts, and thrashing.

The Full Picture: Agent Memory Architecture



> This is the **von Neumann bottleneck of AI agents** — everything must flow through the context window. All memory strategies are about managing what enters and exits this finite space.

Key Takeaways

1. **Agent memory** maps to human memory: working (context), episodic (logs), semantic (facts), procedural (skills)
2. **Prompt engineering** is powerful (4.4x improvement from CoT) but brittle (76% variance from formatting)
3. **Context engineering** supersedes prompt engineering — it's about building **systems** that assemble the right context dynamically
4. **OpenClaw** = general-purpose agent for life automation; **Claude Code** = specialist agent for software engineering
5. Context is **finite** because of $O(n^2)$ attention, KV cache costs, dollar costs, and quality degradation
6. **CLAUDE.md** implements a memory hierarchy for agents — the same caching principles from hardware apply
7. The future of AI agents is better **memory management**, not just bigger context windows

> The future of AI agents is not bigger context windows — it's smarter **memory management**. The same lesson from hardware applies: the fastest memory is the one you don't have to access.

Next Lecture Preview

Coming up: Encoding & Evolution

Now that we understand how agents manage memory, we'll look at:

- How data is encoded for storage and transmission
- Schema evolution and backward/forward compatibility
- Why encoding choices matter for distributed systems

Connection to today: Agent memory must be **serialized** (encoded) for persistence and **deserialized** for retrieval. The encoding format determines how efficiently agents can store and recall information.

References (1/2)

- Brown et al. Language Models are Few-Shot Learners. NeurIPS 2020.
- Wei et al. Chain-of-Thought Prompting Elicits Reasoning. NeurIPS 2022.
- Yao et al. Tree of Thoughts. NeurIPS 2023.
- Yao et al. ReAct: Reasoning and Acting. ICLR 2023.
- Park et al. Generative Agents. UIST 2023.
- Packer et al. MemGPT: LLMs as Operating Systems. 2023.
- Sumers et al. CoALA: Cognitive Architectures for Language Agents. 2023.
- Liu et al. Lost in the Middle. TACL 2024.
- Sclar et al. Sensitivity to Prompt Design. ICLR 2024.

References (2/2)

- Vaswani et al. Attention Is All You Need. NeurIPS 2017.
- Pope et al. Efficiently Scaling Transformer Inference. MLSys 2023.
- Dao et al. FlashAttention. NeurIPS 2022.
- Gu & Dao. Mamba: Linear-Time Sequence Modeling. 2023.
- Khattab et al. DSPy. ICLR 2024.
- Anthropic. Building Effective Agents. Dec 2024.
- Anthropic. Effective Context Engineering for AI Agents. 2025.
- Schmid, P. Context Engineering. 2025.
- Karpathy, A. Context Engineering. June 2025.